SOMFY POE MOTOR API FOR SYSTEM INTEGRATION

COPYRIGHT NOTICE 3
CHANGELOG 3
ABSTRACT 4
DEFINITIONS 4
DATA FLOW 4
Synergy foundation 4
General rules 4
STATUS 5
Status interface 5
Ping method 5
Info method 6
Position method 7
Speeds method 8
Speeds range method 9
Ramps method 10
Ramps range method 11
LED method 12
IP method 13
IPV4 method 14
Diagnostics method 15
MOTOR CONTROL 16
Move interface 16
Stop method 17
Up method 18
Down method 19
To method 20
Wink method 21
Relative method 22
Intermediate position interface 23
Next up method 23

Next down method 24
IP method 25
Group interface 26
Get method 26
Set method 27
Group Control 28
Lock interface 29
Get method 29
Set method 30
SETTINGS 31
Settings interface 31
IP interface 31
Speeds method 33
LED method 34
Name method 35
IPV4 method 36
Ramps method 37
Advanced node discovery 38
Transport 38
UPD 38
TCP over TLS 38
CoAP 38
Security 39
Authentication interface 39
Get the current key of the motor 40
Set the key of the motor 41
Error code 42
Table of errors 42

COPYRIGHT NOTICE

Copyright (C) 2019-2023, Somfy SAS. All rights reserved. All reproduction, use or distribution of this document, in whole or in part, by any means, without Somfy SAS prior written approval, is strictly forbidden.

CHANGELOG

Rev 007 03/01/2023

- Add details in Transport, TCP over TLS section TCP connection targetID & groupID are ignored
- Add a new error message 50 Incorrect

Rev 006 09/10/2021

- Add detail on TargetID
- Add a section for CoAP
- Replace the param groupID by name in the command group.set

Rev 005 06/23/2021

- Add data to status.info reply
- Add diagnostics
- Add new errors message

Rev 004 11/30/2020

- Change the value to restore the default speed from 0 to -1
- Add firmware in info method
- Update status.position with new fields
- Add error code

Rev 003 09/15/2020

- Change group naming
- Add details on security

Rev 002 06/15/2020

- Add security, API to negotiate the encryption key
- How data are encrypted with this key

Rev 001 04/17/2020

- Add example on move interface
- Add requirements
- Integrate the remarks after 1st validation steps

Rev 000 10/29/2019

- Creation

ABSTRACT

This document describes the Somfy Synergy Json-RPC API of a Power Over Ethernet (PoE) motor. This document defines the Synergy messages so that:

- A project implementing Synergy JSon-RPC API shall follow these recommendations
- A project implementing a device interacting with the Synergy JSon-RPC shall use the information here enclosed as a guide on how the device will react.

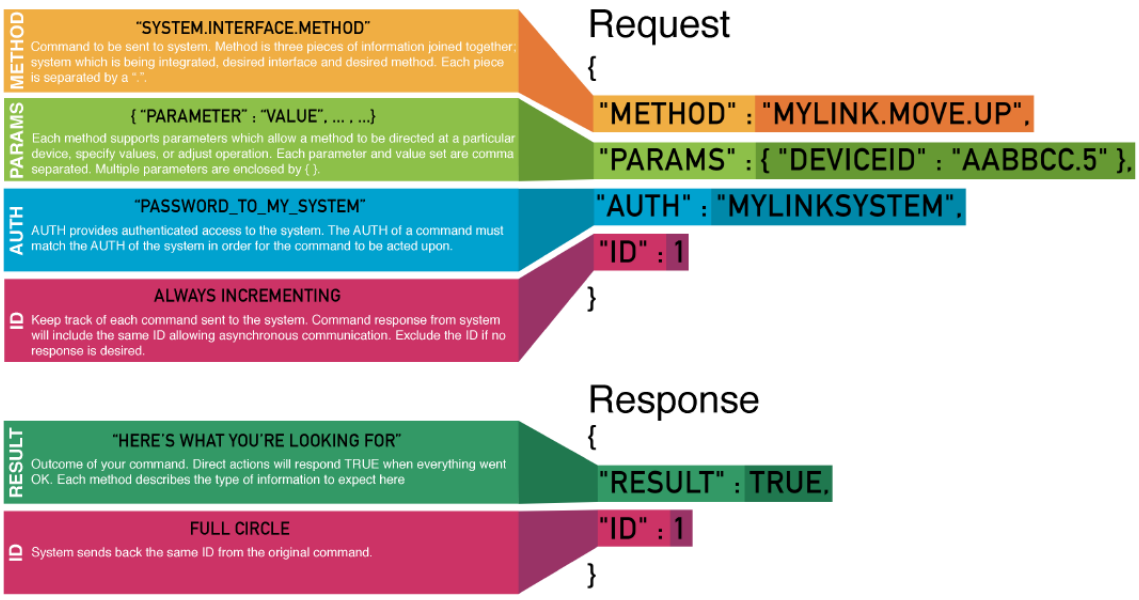
DEFINITIONS

Actuator (or motor)

This is the basic node which controls the end product (blind, screen, etc.).

DATA FLOW

Synergy foundation



General rules

- Unknown method will be not answered
- Unknown parameters are ignored
- targetID is a string that represents a value coded on 24 bits in hexadecimal “AABBCC” , targetID is built with the three least significant bytes of the mac address
- groupID is a string of max 24 ASCII characters long
- If a numerical value is not significative the value can be set to null
- In general, targetID XOR groupID can be used to address a motor XOR a group of motors
- The maximum size of a JSON packet is set to 512 bytes
- All commands have a reply

STATUS

Status Interface

This interface is used to retrieve the status of a motor. Here is the API for the status interface:

```
{
  "description" : "status retrieving interface",
  "path" : "status",
  "interface" : "status",
  "methods" : ["ping", "info", "position", "speeds", "ramps", "led",
"motorheadswitch", "ip", "ipv4" ],
}
```

Ping method

This method is used to check if a device is online. Here is the API of the ping method:

```
{
  "description" : "check if a device is online",
  "path" : "status.ping",
  "params" : [
  {
    "name" : "targetID",
    "type" : "string",
    "default" : "*",
    "description" : "device to ping, use '*' as a wildcard"
  }
  ],
  "returns" :
  {
    {
      "name" : "result",
      "type" : "boolean",
    },
    {
      "name" : "targetID",
      "type" : "string",
      "description" : "recall the identity of the target"
    }
  }
}
```

The reply could be sent up to 400ms after reception.

Example: With 4 actuators replying

REQUEST

```
{ "method" : "status.ping", "params" : { "targetID" : "*"}, "id" : 0 }
```

RESPONSE

```
{"result":true, "id":0, "targetID":"12AB89"}
{"result":true, "id":0, "targetID":"12BC89"}
{"result":true, "id":0, "targetID":"12CD89"}
{"result":true, "id":0, "targetID":"12DE89"}
```

Info method

This method is used to retrieve descriptive information about a device. Here is the API for the info method:

```
{
  "description" : "retrieve information about a device",
  "path" : "status.info",
  "params" : [
    {
      "name" : "targetID",
      "type" : "string",
      "description" : "device to query, use '*' as a wildcard"
    }
  ],
  "returns" : [
  {
    "type" : "array",
    "items" : [
      {
        "description" : "devices information",
        "type" : "object"
      }
    ]
  },
  {
    "type" : "object",
    "description" : "requested information",
    "attributes" : [
      { "name" : "name", "type" : "string" },
      { "name" : "model", "type" : "string" },
      { "name" : "hostname", "type" : "string" },
      { "name" : "synergy_version", "type" : "string" },
      { "name" : "targetID", "type" : "string"},
      { "name"  : "firmware", "type" : "string"},
      { "name"  : "hardware", "type" : "string"},
      { "name"  : "mac", "type" : "string"},
      { "name"  : "firmwareupgrade", "type" : "string" }    ]
  }
  ],
}
```

Example: Possible values for model, Sonesse 30 PoE, Sonesse 40 PoE, and Curtain PoE

{ "name" : "firmwareupgrade", "type" : "string" }* is optional, if a firmware upgrade is in progress the value of the progression will display in %

REQUEST

```
{ "method" : "status.info", "params" : { "targetID" : "AABBCC" }, "id" : 1
}
```

RESPONSE

```
{"result': true, 'targetID': 'AABBCC', 'id': 1, 'info':{'name': 'Blind west
room #123', 'model': 'Sonesse 30 PoE', 'synergy_version': 5, 'firmware':
'5156432A07/5155239A10', 'hardware': 'Undefined/5153582A000', 'mac':
'4C:C2:06:16:00:26', 'hostname': 'sfy_poe_000114'}}
```

Position method

This method is used to retrieve the position of a motor. Here is the API for the position method:

```
{
  "description" : "retrieve an actuator position",
  "path" : "status.position",
  "params" : [
    {
      "name" : "targetID",
      "type" : "string"
    }
  ]
  "returns" :
{
  "type" : "object",
  "position" : {
    { "name" : "value", "type", "number" },
    { "name": "status", "type" : ["stopped", "running", "blocked"]
},
    { "name": "direction", "type" : ["down / close", "up / open"]},
    { "name": "source", "type" : ["internal", "network message",
"local ui", "unknown"] },
    { " name ": "cause", "type" : ["target reached", "explicit
command", "obstacle detection", "over-current protection", "thermal
protection", "unknown"]}  }
}
```

If the two end limits are not set then the position shall be set to null.

Set “source” to “internal” in case the orders come from the following sources:

- Limit / IP / Position reached, Over-current, obstacle detection, thermal protection
- Set to “network message” any message receive
- Set to local UI when orders coming from an action on the button, or the embedded webGUI
- Unknown after power up no movement performed yet

The decimal part is 3 digits after the decimal point: XX.xxx %

Example: Replied position of 63.512 %

REQUEST

```
{ "method" : "status.position", "params" : { "targetID" : "AABBCC" }, "id"
: 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "position" : { "value" :
63.512, "status" : "stopped", "direction" : "down/ close", "source" :
"internal", "cause" : "target reached" }}
```

Speeds method

This method is used to retrieve the speed of a motor. Here is the API for the speed method:

```
{
  "description" : "retrieve actuator's speeds",
  "path" : "status.speeds",
  "params" : [
    {
      "name" : "targetID",
      "type" : "string"
    }
  ],
  "returns" : {
  "type" : "object",
  "speeds" : {
    { "up" : "number", "description" : "Up speed in rpm" },
    { "down" : "number" "description" : "Down speed in rpm" },
    { "slow" : "number", "description" : "Slow speed in rpm" }
    }
  }
}
```

The resolution of the speed is ¼ rpm.

Example:

REQUEST

```
{ "method" : "status.speed", "params" : { "targetID" : "AABBCC" }, "id" : 1
}
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "speeds" : {"up" : 25,
"down" : 25, "slow" : 15} }
```

Speeds range method

This method is used to retrieve the speed range of a motor. Here is the API for the speed range method:

```
{
  "description" : "retrieve actuator's speeds range",
  "path" : "status.speeds_range",
  "params" : [
    {
      "name" : "targetID",
      "type" : "string"
    }
  ],
  "returns" : {
  "type" : "object",
  "speeds" : {
    { "min" : "number", "description" : "Minimum speed in rpm" },
    { "max" : "number" "description" : "Maximum speed in rpm" },
    { "default" : "number", "description" : "Default for nominal
speed in rpm" },
    { "default_slow" : "number", "description" : "Default for slow
speed in rpm" }
    }
  }
}
```

Example:

REQUEST

```
{ "method" : "status.speeds_range", "params" : { "targetID" : "AABBCC" },
"id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "speeds_range" : {"min" :
10, "max" : 28, "default" : 25, "default_slow" : 15 } }
```

Ramps method

This method is used to read the ramps or soft start and soft stop values of a motor. Here is the API for the ramps method:

```
{
  "description" : "Read the ramps value",
  "path" : "status.ramps",
  "params" : [
    {
      "name" : "targetID",
      "type" : "string"
    }
  ],
  "returns" : {
  "type" : "object",
  "ramps" : {
    { "softstartup" : "number", "description" : "Soft start in up
direction (ms)" },
    { "softstartdown" : "number", "description" : "Soft start in
down direction (ms)" },
    { "softstopup" : "number", "description" : "Soft stop in up
direction (ms)" },
    { "softstopdown" : "number", "description" : "Soft stop in down
direction (ms)" },
    }
  }
}
```

Example:

REQUEST

```
{ "method" : "status.ramps", "params" : { "targetID" : "AABBCC" }, "id" : 1
}
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "ramps" : {"softstartup" :
25, "softstartdown" : 25, "softstopup" : 15, "softstopdown" : 15} }
```

The value 0 for the ramp means deactivated.

Ramps range method

This method is used to read the ramps range or soft start and soft stop range values of a motor. Here is the API for the ramps method:

```
{
  "description" : "Read the ramps range value",
  "path" : "status.ramps_range",
  "params" : [
    {
      "name" : "targetID",
      "type" : "string"
    }
  ],
  "returns" : {
  "type" : "object",
  "ramps.range" : {
    { "min" : "number", "description" : "Minimal value for the ramp
in (ms)" },
    { "max" : "number", "description" : "Maximal value for the ramp
in (ms)" },
    { "default" : "number", "description" : "Default value for the
ramps in (ms)" }
    }
  }
}
```

Example:

REQUEST

```
{ "method" : "status.ramps_range", "params" : { "targetID" : "AABBCC" },
"id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "ramps_range" : {"min" :
500, "max" : 2500, "default" : 1500} }
```

LED method

This method is used to retrieve the LED state of a motor. Here is the API for the led method:

```
{
  "description" : "retrieve the led's state of the actuator",
  "path" : "status.led",
  "params" : [
    {
      "name" : "targetID", "type" : "string"
    }
  ],
  "returns" : {
  "name" : "state",
  "type" : "boolean",
  "description" : "led feedback enable if true, disable otherwise"
  }
}
```

Example: Replied LED is enabled

REQUEST

```
{ "method" : "status.led", "params" : { "targetID" : "AABBCC" }, "id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "state" : true  }
```

IP method

This method is used to get the intermediate positions of a motor. Here is the API for the ip method:

```
{
  "description" : "retrieve the intermediates positions",
  "path" : "status.ip",
  "params" : [
    {
      "name" : "targetID", "type" : "string"
    }
  ],
  "returns" : {
  "type" : "object",
  "name" : "ip"
  {
    { "1" : "number", "description" : "Value in percent of ip #1" },
    { "2" : "number", "description" : "Value in percent of ip #2" },
    { "3" : "number", "description" : "Value in percent of ip #3" },
    { "4" : "number", "description" : "Value in percent of ip #4" },
    …
    { "15" : "number", "description" : "Value in percent of ip #15"
},
    { "16" : "number", "description" : "Value in percent of ip #16"
}
  }
}
```

If the IP is not set the value is null.

Example: Replied LED is enabled

REQUEST

```
{ "method" : "status.ip", "params" : { "targetID" : "AABBCC" }, "id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "ip" : {"1" : "10.23", "2"
: 45.236, "3" : null, "4" : 67.9, "5" : "87.4", "6" : "42.3", "7" : null,
"8" : null, "9" : null, "10" : null, "11" : null, "12" : null, "13" : null,
"14" : null, "15" : null, "16" : null} }
```

IPV4 method

This method is used to retrieve the internet protocol settings of a motor. Here is the API for the ipv4 method:

```
{
  "description" : "retrieve the motor ipv4 settings",
  "path" : "status.ipv4",
  "params" : [
    {
      "name" : "targetID", "type" : "string"
    }
  ],
  "returns" : [
    { "name" : "result", "type" : "boolean"},
    { "name" : "targetID", "type" : "hex24"},
    { "name" : "ipv4", "type" : {
      { "name" : "dhcp", "type" : "bool", "description" : "true dhcp
is enabled, false dhcp is disabled" },
      { "name" : "ipaddress", "type" : "string" },
      { "name" : "subnetmask", "type" : "string" },
      { "name" : "gateway", "type" : "string" },
      { "name" : "dns1", "type" : "string" },
      { "name" : "dns2", "type" : "string" }
    }
  ]
}
```

To avoid issues the groupID target is not accepted for this command.

Example:

REQUEST

```
{ "method" : "status.ipv4", "params" : { "targetID" : "AABBCC" }, "id" : 1
}
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "ipv4" : {"dhcp" : true,
"ipaddress" : "192.168.1.10", "subnetmask" : "255.255.255.0", "gateway" :
"192.168.1.1", "dns1" : "192.168.1.1", "dns2" : "192.168.1.1" } }
```

Diagnostics method

This method is used to retrieve the diagnostic counters of a motor. Here is the API for the diagnostics method:

```
{
  "description" : "retrieve the diagnostics of the motor",
  "path" : "status.diagnostics",
  "params" : [
    {
      "name" : "targetID", "type" : "string"
    },
    {
      "counter" : "Counter Name", "type" : "string"
    }
  ],
  "returns" : [
    { "name" : "result", "type" : "boolean"},
    { "name" : "targetID", "type" : "hex24"},
    { "name" : "counter", "number" }
  ]
}
```

	Counter description	Counter Name
Motor	Power On reset counter MCU1	power_on_reset
	Number of output shaft revolutions (CW)	output_shaft_revolutions_cw
	Number of output shaft revolutions (CCW)	output_shaft_revolutions_ccw
	Number of stop at the DOWN/CLOSE limit	stop_at_down_limit
	Number of stop at the UP/OPEN limit	stop_at_up_limit
	Number of running time stop	running_time_stop
	Number of stop caused by an obstacle	stop_by_obstacle
	Number of UP/OPEN end limit	up_endlimit_adjustment
	Number of DOWN/CLOSE end limit adjustment	down_endlimit_adjustment
	Number of return to factory mode	back_to_factory_mode
	Number of power cut	power_cut
	Number of power cut during a movement	power_cut_during_movement
	Max voltage detected	max_voltage_detected
	Current position	current_position
	Rotation Direction	rotation_direction
	Value of the UP/OPEN end limit	up_endlimit_value
	Value of the DOWN/CLOSE end limit	down_endlimit_value
	Value of the first favorite position	first_favorite_position
	Number of thermal stop	thermal_stop
	Current motor mode	current_motor_mode
Media	Number of Synergy UDP requests received	udp_rx_requests
	Number of Synergy UDP requests sent	udp_tx_requests
	Number of Synergy TCP requests received	tcp_rx_requests
	Number of Synergy TCP requests sent	tcp_tx_requests
	Number of Synergy COAP requests received	coap_rx_requests
	Number of Synergy COAP requests sent	coap_tx_requests
	Number of IP address assigned via DHCP	dhcp_address_assigned
	Number of connexion on WebGUI	webgui_connection

Example:

REQUEST

```
{ "method" : "status. diagnostics", "params" : { "targetID" : "AABBCC",
"counter" : "power_cut" }, "id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1, "counter" : 42 }
```

MOTOR CONTROL

All motor control interfaces can move, lock, and unlock the motor.

All motor control interfaces can move in a group.

Move interface

This interface is used to move a motor. It has one sub-interface called ”ip” later described in this document. Here is the API for the move interface:

```
{
  "description" : "actuator moving interface",
  "path" : "move",
  "interface" : "move",
  "methods" : [ "stop", "up", "down", "to", "wink"],
  "objects" : [ "ip" ]
}
```

Generality on the “seq” parameter

- By adding this parameter to a movement method, the actuator will periodically send the position with the requested “seq” id
- The range of this parameter is [1-65535]
- The period of sending of the position is implementation dependent but should be lesser than 1,1s

"lock" & "priority" parameters:

All the move commands accept these optional parameters are to manage the lock of the actuator.

When the "lock" is set to true and "priority" set, the actuator will lock to the priority level specified then move to the target. "lock" set to false is ignored in the move interface.

When only the "priority" is defined, the command will be executed only if the priority is higher than the current priority lock. When no priority is defined the priority is the lowest.

Stop method

This method is used to stop a movement in progress of a motor. Here is the API for the stop method:

```
{
  "description" : "stop current movement",
  "path" : "move.stop",
  "params" : [
    { "name" : "targetID", "type" : "string", "default": "*" },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) },
  ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

Example:

REQUEST

```
{ "method" : "move.stop", "params" : { "targetID" : "EEFFDD" }, "id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID" : "EEFFDD", "id" : 1  }
```

Up method

This method is used to move a motor in the UP direction. Here is the API for the up method:

```
{
  "description" : "move actuator in up direction",
  "path" : "move.up",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1) },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "seq", "type" : "number", "optional": true(2) },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) },
 ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

(2): The “seq” parameter forces an update of the current position periodically.

Example:

REQUEST

```
{ "method" : "move.up", "params" : { "targetID" : "12AB89", "seq" : "1" },
"id" : 2 }
```

RESPONSE

```
{"result":true, "id":2, "targetID":"12AB89"}
{"method":"status.position","params":{"position":8, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":6, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":3, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":0, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":0, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":0, "seq":1,
eot":true},"targetID":"12AB89"}
```

The last reply adds the parameter “eot” set to true.

(3): When parameter “lock” is present “priority” is mandatory.

Down method

This method is used to move a motor in the DOWN direction. Here is the API for the down method:

```
{
  "description" : "move actuator in down direction",
  "path" : "move.down",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1)
},
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "seq", "type" : "number", "optional": true(2) },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }
 ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

(2): The “seq” parameter forces an update of the current position periodically.

Example:

REQUEST

```
{ "method" : "move.down", "params" : { "targetID" : "12AB89", "seq" : "1"
}, "id" : 2 }
```

RESPONSE

```
{"result":true, "id":2, "targetID":"12AB89"}
{"method":"status.position","params":{"position":92,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":94,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":97,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":100, "seq":1},
"targetID": "12AB89"}
{"method":"status.position", "params":{"position":100, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":100, "seq":1, "eot":true},
"targetID": "12AB89"}
```

The last reply adds the parameter “eot” set to true.

(3): When parameter “lock” is present “priority” is mandatory.

To method

This method is used to move a motor to an absolute position. Here is the API for the to method:

```
{
  "description" : "move the actuator to an absolute position",
  "path" : "move.to",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1) },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "position", "type" : "number", "description" : "Unit in percent"
},
    { "name" : "seq", "type" : "number", "optional": true(2) },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }
 ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

(2): The “seq” parameter forces an update of the current position periodically.

Example 1:

REQUEST

```
{ "method" : "move.to", "params" : { "targetID" : "EEFFDD", "position" : "42.5" }
```

RESPONSE

```
{ "result" : true}, "targetID": " EEFFDD" }
```

Example 2: Using the optional “seq” parameter

REQUEST

```
{ "method" : "move.to", "params" : { "targetID" : "12AB89", "position" :
"42.5", "seq" : "1" }, "id" : 2 }
```

RESPONSE

```
{"result":true, "id":2, "targetID":"12AB89"}
{"method":"status.position", "params":{"position":33, "seq":1}, "targetID":
"12AB89"}
{"method":"status.position", "params":{"position":36, "seq":1}, "targetID":
"12AB89v}
{"method":"status.position", "params":{"position":39, "seq":1},
"targetID":"12AB89"}
{"method":"status.position", "params":{"position":42.5, "seq":1},
"targetID":"12AB89"}
{"method":"status.position", "params":{"position":42.5, "seq":1},
"targetID":"12AB89"}
{"method":"status.position", "params":{"position":42.5, "seq":1,
"eot":true},"targetID":"12AB89"}
```

The last reply adds the parameter “eot” set to true.

(3): When parameter “lock” is present “priority” is mandatory.

Wink method

This method is used to identify or wink a motor a short movement in both directions. Here is the API for the wink method:

```
{
  "description" : "Short movement in both direction",
  "path" : "move.wink",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1) },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }
  ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

Example 1: For a motor with targetID EEFFDD

REQUEST

```
{ "method" : "move.wink", "params" : { "targetID" : "EEFFDD" } }
```

RESPONSE

```
{ "result" : true, "targetID":"EEFFDD"  }
```

Example 2: For the group with groupID “Room 012345”, 2 motors in the group targetID "12AB89" & "12AB8A"

REQUEST

```
{ "method" : "move.wink", "params" : { "groupID" : "Room 012345" } }
```

RESPONSE

```
{ "result" : true, "targetID":"12AB89" }
{ "result" : true, "targetID":"12AB8A" }
```

(2): When parameter “lock” is present “priority” is mandatory.

Relative method

This method is used to move a motor to a relative position. Here is the API for the relative method:

```
{
  "description" : "Relative move to given position",
  "path" : "move.relative",
  "params" : [
    { "name" : "targetID", "type" : "string"(1), "optional":
true(1)},
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "duration", "type" : "integer", "description" :
"relative movement in ms from the current position" },
    { "name" : "direction", "type" : "string", "values" : ["up",
"down"] },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }
  ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

Example 1: For an actuator with targetID EEFFDD, relative move of 800ms in up direction

REQUEST

```
{ "method" : "move.relative ", "params" : { "targetID" : "EEFFDD",
"duration" : 800, "direction" : "up" } }
```

RESPONSE

```
{ "result" : true, "targetID":"EEFFDD"  }
```

Example 2: For the group with groupID “Room 012345”, 2 motors in the group targetID "12AB89" & "12AB8A"

REQUEST

```
{ "method" : "move.relative", "params" : { "groupID" : "Room 012345" } }
```

RESPONSE

```
{ "result" : true, "targetID":"12AB89" }
{ "result" : true, "targetID":"12AB8A" }
```

(2): When parameter “lock” is present “priority” is mandatory.

Intermediate position interface

This interface is used to move the motor to an intermediate position. Here is the API for the ip interface:

```
{
  "description" : "actuator intermediate position interface",
  "path" : "move.ip",
  "interface" : "move.ip",
  "methods" : [ "nextup, "nextdown" ],
}
```

Next up method

This method is used to move the motor to the next intermediate position up direction. Here is the API of the nextup method:

```
{
  "description" : "move to next ip position",
  "path" : "move.ip.nextup",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1) },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "seq", "type" : "number", "optional": true },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }
  ],
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

Example:

REQUEST

```
{ "method" : "move.ip.nextup", "params" : { "targetID" : "EEFFDD" }, "id" :
2 }
```

RESPONSE

```
{ "result" : true, "id":2, "targetID":"EEFFDD"} }
```

(2): When parameter “lock” is present “priority” is mandatory.

Next down method

This method is used to move the motor to the next intermediate position down direction. Here is the API for the nextdown method:

```
{
  "description" : "move to the next ip down direction",
  "path" : "move.ip.nextdown",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1) },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "seq", "type" : "number", "optional": true },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }
  ]
  "returns" : { "type" : "boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

Example:

REQUEST

```
{ "method" : "move.ip.nextdown", "params" : { "targetID" : "EEFFDD" }, "id"
: 2 }
```

RESPONSE

```
{ "result" : true, "id":2, "targetID":"12AB89"} }
```

(2): When parameter “lock” is present “priority” is mandatory.

IP method

This method is used to move the motor to a desired intermediate position. Here is the API for the ip method:

```
{
  "description" : "move to given ip position",
  "path" : "move.ip",
  "params" : [
    { "name" : "targetID", "type" : "string", "optional": true(1) },
    { "name" : "groupID", "type" : "string", "optional": true(1) },
    { "name" : "num", "type" : "integer", "Boolean" "Description" : "Index
of the IP from 1 to 16" },
    { "name" : "priority", "type" : "number", "optional": true },
    { "name" : "lock", "type" : "boolean", "optional": true(3) }

  ],
  "returns" : { "type" : "Boolean" }
}
```

(1): A targetID XOR groupID is mandatory.

Example 1: Move the motor to the given intermediate position

REQUEST

```
{ "method" : "move.ip", "params" : { "targetID" : "EEFFDD", "num" : 1 },
"id" : 2 }
```

RESPONSE

```
{ "result" : true, "id":2, "targetID":"12AB89"} }
```

Example 2: With the “seq” parameter, supposed than the IP #1 is 42.5%

REQUEST

```
{ "method" : "move.ip", "params" : { "targetID" : "12AB89", "num" : 1,
"seq" : 1 }, "id" : 2 }
```

RESPONSE

```
{"result":true, "id":2, "targetID":"12AB89"}
{"method":"status.position", "params":{"position":33,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":36,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":39,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":42.5,
"seq":1},"targetID":"12AB89"}
{"method":"status.position", "params":{"position":42.5, "seq":1},
"targetID":"12AB89"}
{"method":"status.position","params":{"position":42.5, "seq":1,
"eot":true},"targetID":"12AB89"}
```

Group interface

This interface is used to manage and control groups of devices. Here is the API for the group interface:

```
{
  "description" : "group related interface",
  "path" : "group",
  "interface" : "group",
  "methods" : ["get", "set"]
}
```

Get method

This method is to list which groups a motor is member of. Here is the API of the get method:

```
{
  "description" : "get the list of groups a motor is in",
  "path" : "group.get",
  "params" : [
    { "name" : "targetID", "type" : "string" },
    {
      "name" : "index",
      "type" : "integer",
      "default" : -1,
      "description" : "The index of the groupID to retrieve , -1 to
retrieve everything"
    }
  ],
  "returns" : [
    { "name" : "result", "type" : "boolean" },
    {
      "name" : "group"
      "type" : "array",
      "items" : [
    {
      "name" : "index"
      "description" : "groupID",
      "type" : "string"
    }
  ],
  "description" : "This reply will be used when requesting all the groups"
  },
  {
    "type" : "string",
    "description" : "retrieved groupID"
  }
  ]
}
```

IDs are from 0 to the number of groups minus 1. 16 group IDs will range from 0 to 15.

Example 1: Retrieve all the groups IDs where the motor “AABBCC” is assigned 16 groups

REQUEST

```
{ "method" : "group.get", "params" : { "targetID" : "AABBCC", "index" : -1} }
```

RESPONSE

```
{ "result" : true, "group" :["DDEEFF", "001122", "334455", "667788",
"667788", "99AABB", "CCDDEE", "FF0008", "FF0009", "FF000A", "FF000B",
"FF000C", "FF000D", "FF000E", "FF000F", "FF0010"], "targetID" : "AABBCC"}
```

Example 2: Retrieve the group ID where motor “AABBCC” is assigned to group 3

REQUEST

```
{ "method" : "group.get", "params" : { "targetID" : "AABBCC", "index" : 2} }
```

RESPONSE

```
{ "result" : true, "group" : [ "334455" ], "targetID" : "AABBCC" }
```

Set method

This method is to add a motor into a group. Here is the API for the set method:

```
{
  "desription" : "register a motor in a group",
  "path" : "group.set",
  "params" : [
  { "name" : "targetID", "type" : "string" },
  {
    "name" : "index",
    "type" : "integer",
    "description" : "The index of the groupID to set"
  },
  {
    "name" : "name",
    "type" : "string",
    "description" : "The name to program, use an empty string to erase the
group"
  }
  ],
  "returns" : { "type" : "boolean" }
}
```

"" The empty string is the default value.

Example 1: Add motor “AABBCC” to group index 1 groupID “Room DDEEFF”

REQUEST

```
{ "method" : "group.set", "params" : { "targetID" : "AABBCC", "index" : 1,
"groupID" : "Room DDEEFF" } }
```

RESPONSE

```
{"result":true, "id":0, "targetID":"AABBCC"}
```

Example 2: Remove motor “AABBCC” from group index 3

REQUEST

```
{ "method" : "group.set", "params" : { "targetID" : "AABBCC", "index" : 3,
"groupID" : "" } }
```

RESPONSE

```
{ "result" : true, "targetID" : "443322" }
```

Group Control

See move interface; replace targetID with groupID.

Example: Move up all the actuators inside the group “Room DDEEFF”

REQUEST

```
{ "method" : "move.up", "params" : { "groupID" : "Room DDEEFF" } }
```

RESPONSE

```
{ "result" : true, "targetID" : "AABBCC" }
{ "result" : true, "targetID" : "DDEEFF" }
{ "result" : true, "targetID" : "443322" }
```

Lock interface

This interface is used to manage device locks. Here is the API for the lock interface:

```
{
  "description" : "lock related interface",
  "path" : "lock",
  "interface" : "lock",
  "methods" : ["get", "set" ],
  "objects" : ["move"]
}
```

Get method

This method is used to retrieve the lock state of a motor. Here is the API of the lock get method:

```
{
  "desription" : "get the lock state of a motor",
  "path" : "lock.get",
  "params" : [
    { "name" : "targetID", "type" : "string" }
    ],
  "returns" : {
  "type" : "object",
  "description" : "lock information",
  "attributes" : [
    { "name" : "state", "type" : "boolean" },
    { "name" : "priority", "type" : "integer" },
    { "name" : "src", "type" : "string 24"}
    ]
  }
}
```

The “priority” is set in the result only when the state is true. An optional “src” attribute may be set in the result, providing the ID of the device that activated the lock.

Example 1: Motor not locked response

REQUEST

```
{ "method" : "lock.get", "params" : {"targetID" : "AABBCC"}, "id" : "2" }
```

RESPONSE

```
{ "result" : true, "lock" : false }
```

Example 2: Motor locked response

RESPONSE

```
{ "result" : true, "lock" : true, "priority" : 100 }
```

Set method

This method is used to set the lock state of a motor. Here is the API for the lock set method:

```
{
  "desription" : "lock or unlock a motor",
  "path" : "lock.set",
  "params" : [
    { "name" : "targetID", "type" : "string" },
    {
      "name" : "state",
      "type" : "boolean",
      "description" : "requested lock state"
    },
    {
      "name" : "priority",
      "type" : "integer",
      "description" : "Priority range from 1 - 2^32-1, highest the
number, highest the priority"
    },
    {
      "name" : "src",
      "type" : "string",
      "description" : "Name to identify the source of lock, if src
is empty the IP will be used"
      "size" : 20
  ],
  "returns" : { "type" : "boolean" }
}
```

The motor will continue its movement, to lock a motor at a given position see move interface. Once the lock is set to true with priority P, the lock shall be removed only by setting a priority greater or equal to P.

Example:

REQUEST

```
{ "method" : "lock.set", "params" : {"targetID" : "AABBCC", "state" : true,
"priority" : 1000 } }
```

RESPONSE

```
{ "result" : true, "targetID" : "AABBCC" }
```

SETTINGS

Settings interface

This interface is used to retrieve the settings status of a motor. Here is the API of the status interface:

```
{
  "description" : "status retrieving interface",
  "path" : "settings",
  "interface" : "settings",
  "methods" : ["ip", "speed", "led", "motorheadswitch", "name", "ipv4",
"ramps"]
}
```

IP interface

This interface is used to set the intermediate positions of a motor. Here is the API for the ip interface:

```
{
  "description" : "Set the motor's intermediate positions",
  "path" : "settings.ip",
  "params" : [
    { "name" : "targetID", "type" : "string" },
    { "name" : "values" : [
      { "1" : "number", "description" : "Set the ip #1 with the
percentage value, -1 set the ip at current position", "optional" :
true },
      { "2" : "number", "optional" : true },
      { "3" : "number", "optional" : true },
      { "4" : "number", "optional" : true },
      { "5" : "number", "optional" : true },
      { "6" : "number", "optional" : true },
      { "7" : "number", "optional" : true },
      { "8" : "number", "optional" : true },
      { "9" : "number", "optional" : true },
      { "10" : "number", "optional" : true },
      { "11" : "number", "optional" : true },
      { "12" : "number", "optional" : true },
      { "13" : "number", "optional" : true },
      { "14" : "number", "optional" : true },
      { "15" : "number", "optional" : true },
      { "16" : "number", "optional" : true }
      ]
    },
    { "setautomatically" : "number", "optional" : true,
"description" : "Set automatically n intermediate positions"},

  ],
  "returns" : { "type" : "boolean" }
}
```

Use the null value to erase an intermediate position.

In case of setting more than one IP at a time, if false is received in return, the user shall get the value of all IPs with the command status.ip.

Example 1: Set IP 3 and 6 to 25.6 and 70.1% respectively

REQUEST

```
{ "method" : "settings.ip", "params" : {  "targetID" : "EEFFDD", "values" : [ { "3" : 25.6 }, { "6" : 70.1 } ] }
```

RESPONSE

```
{ "result" : true, "targetID" : "EEFFDD" }
```

Example 2: Set 3 equally spaced IPs

REQUEST

```
{ "method" : "settings.ip", "params" : {  "targetID" : "EEFFDD", "values" : [ { " setautomatically" : 3} ] } }
```

RESPONSE

```
{ "result" : true, "targetID" : "EEFFDD" }
```

The actuator will remove all the intermediate positions and set 3 IPs:

- IP 1 to 25%
- IP 2 to 50%
- IP 3 to 75%

Use the special value -1 to set the IP at the current position.

Speeds method

This method is used to set the speed of a motor. Here is the API for the speeds method:

```
{
  "description" : "Set the speed of the actuators",
  "path" : "settings.speeds",
  "params" : [
    { "name" : "targetID", "type" : "string" },
    { "name" : "up", "type" : "number", "description" : "Angular
speed of the motor in rpm in up direction" },
    { "name" : "down", "type" : "number", "description" : "Angular
speed of the motor in rpm in down direction" },
    { "name" : "slow", "type" : "number", "description" : "Angular
speed of the motor in rpm in slow mode" }
  ],
  "returns" : { "type" : "boolean" }
}
```

At least one of these "up", "down" or "slow" parameters shall be present in the message.

The value of speed -1 restores the default speed of the motor.

Example:

REQUEST

```
{ "method" : "settings.speeds", "params" : {  "targetID" : "EEFFDD", "up" :
23.25, "down" : 18.4, "slow" : 10}
```

RESPONSE

```
{ "result" : true, "targetID" : "EEFFDD"  }
```

Values are rounded to the nearest ¼ rpm.

LED method

This method is used to enable or disable the motorhead LED feedback. Here is the API for the led method:

```
{
  "description" : "Enable or Disable the LED on the motors head",
  "path" : "settings.led",
  "params" : [
    { "name" : "targetID", "type" : "string" },
    { "name" : "state", "type" : "boolean", "description" : "true
enable the LED on the motor head, false disable it" }
  ],
  "returns" : { "type" : "boolean" }
}
```

This only enables or disables the bus communication indicator.

The status indicator will always remain enabled.

Example:

REQUEST

```
{ "method":"settings.led", "params" : {"targetID": "AABBCC", "state":true}}
```

RESPONSE

```
{ "result" : true, "targetID" : "AABBCC" }
```

Name method

This method is used to assign a name to the motor. Here is the API for the name method:

```
{
  "description" : "Set the motor's name",
  "path" : "settings.name",
  "params" : [
    { "name" : "targetID", "type" : "string"  },
    { "name" : "value", "type" : "string" }
  ],
  "returns" : { "type" : "boolean" }
}
```

This sets the motor name label. The maximum name length is 40 characters.

Example:

REQUEST

```
{ "method" : "settings.name", "params" : {"targetID" : "AABBCC", "value" :
"new_motor_name"}, "id" : 0 }
```

RESPONSE

```
{ "result" : true, "id": 0, "targetID" : "AABBCC" }
```

IPV4 method

This method is used to assign the internet protocol parameters to the motor. Here is the API for the ipv4 method:

```
{
  "description" : "Set the IPv4 parameter",
  "path" : "settings.ipv4",
  "params" : [
    { "name" : "targetID", "type" : "string"},
    { "name" : "dhcp", "type" : "bool", "default" : true },
    { "name" : "ipaddress", "type" : "string" },
    { "name" : "subnetmask", "type" : "string" },
    { "name" : "gateway", "type" : "string" },
    { "name" : "dns1", "type" : "string" },
    { "name" : "dns2", "type" : "string" }
  ],
  "returns" : [
    { "name" : "result", "type" : "boolean" },
    { "name" : "targetID", "type" : "string" },
  ]
}
```

If the “dhcp” parameter is set to false, then the “ipaddress”, “subnetmask”, “gateway”, “dns1”, “dns2” parameters are mandatory, otherwise are optional and ignored.

Exception: targetID with parameter “*” shall be rejected when the “dhcp” parameter is set to false (static IP configuration).

Example:

REQUEST

```
{ "method":"settings.ipv4", "params" : {"targetID": "AABBCC", "dhcp":false,
"ipaddress" : "192.168.0.100", "subnetmask" : "255.255.255.0", "gateway" :
"192.168.0.1", "dns1" : "192.168.0.1", "dns2" : "8.8.8.8"}}
```

RESPONSE

```
{ "result" : true, "targetID" : "AABBCC" }
```

Ramps method

This method is used to set the soft start and soft stop duration of the motor. Here is the API for the ramps method:

```
{
  "description" : "retrieve the ramps setting of the actuator",
  "path" : "settings.ramps",
  "params" : [
    {
      "name" : "targetID", "type" : "string"
    }
  ],
  "returns" : {
  "type" : "object",
  "items" : [
    { "name" : "softstartup", "type" : "integer", "description" :
"Soft start up duration in ms" },
    { "name" : "softstartdown", "type" : "integer", "description" :
"Soft start down duration in ms"},
    { "name" : "softstopup", "type" : "integer", "description" :
"Soft stop up duration in ms” },
    { "name" : "softstopdown", "type" : "integer", "description" :
"Soft stop down duration in ms” }
    ]
  }
}}
```

Special value of 0 disables soft start and soft stop and -1 restores the default value.

Example:

REQUEST

```
{ "method" : "settings.ramps", "params" : { "targetID" : "AABBCC",
"softstartup": 800, "softstartdown":800, "softstopup":800, "softstopdown":800
}, "id" : 1 }
```

RESPONSE

```
{ "result" : true, "targetID": "AABBCC", "id":1 }
```

Advanced node discovery

[TBD]

SDDP: Control4

mDNS candidate

Transport

UPD

The actuator listens for synergy commands on port 55055.

It will reply in unicast on UDP port 55055 of the sender.

TCP over TLS

It is possible to use all the commands with TCP over TLS on port 55056, this service is mainly use to set the encryption key but it also supports all commands described above.

With a direct connection the field targetID & groupID are ignored.

CoAP

The standard port used for CoAP is 5683.

CoAP is allowed to transport all the commands that can be sent through raw UDP, except the support of “seq” parameter.

Send the synergy payload associated with CoAP PUT command:

Example with host sfy_poe_000101:

PUT coap://sfy_poe_000101/synergy

Refer to the “Set the key of the motor” section to see the AES-CBC-128 encryption of the payload.

The requests shall be non-confirmable.

Security

Only an authorized client can communicate with the motor.

All Synergy data sent over UDP shall be encrypted:

- Encrypted with AES128 for AFL
- To be compliant with the security requirement
- Transport Synergy over UDP encrypted in AES 128.

The encryption key could be set / get by establishing a secure connection through TCP/TLS using an authentication key printed on the motor label (Could be embedded in a QR Code with mac address).

The first step is to establish a secure connection on the TCP + TLS on port 55056.

The client shall verify the authenticity of the server certificate using the CA (Certificate Authority) provided.

The client shall provide a valid certificate to the server. The certificate will be provided.

Then the client shall authenticate by passing the 4-digit PIN code printed on the motor label.

Once this procedure is complete the client shall have full access to the API described below, otherwise the client shall be disconnected [tbc].

Authentication interface

This interface is used for the security settings of the motor. Here is the API for the authentication interface:

```
{
  "description" : "Security settings",
  "path" : "key ",
  "interface" : "security",
  "methods" : ["get", "set", "auth"]
}
```

```
{
  "description" : "Authenticate with the 4 digits code",
  "path" : "security.auth",
  "params" : [
    { "name" : "code", "type" : "string"},
  ],
  "returns" : { "type" : "boolean", "description" : "true in case of
the code match with the code programmed, false otherwise" }
}
```

Example 1: Entering the correct Motor PIN code “1234”

REQUEST

```
{ "method" : "security.auth", "params" : { "code" : "1234" }, "id" : 1 }
```

RESPONSE

```
{ "result" : true, "id" : 1   }
```

Example 2: Entering an incorrect Motor PIN code

REQUEST

```
{ "method" : "security.auth", "params" : { "code" : "0003" }, "id" : 1 }
```

RESPONSE

```
{ "result" : false, "id" : 1   }
```

After 3 consecutive false answers with an incorrect PIN code, this command will systematically answer false to slow down brute force attack.

Get the current key of the motor

This method is used to get the security key of the motor. Here is the API for the security get method:

```
{
  "description" : "Get the current key of the motor",
  "path" : "security.get"
  "returns" : : {
  "type" : "object",
  "description” : "Key",
  "attributes" : [
    { "name" : "key", "type" : "array of 16 bytes" },
    ]
  }
}
```

Example:

REQUEST

```
{ "method" : "security.get", "id" : 1 } }
```

RESPONSE

```
{ "result" : true, "id" : 1, "key" : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
12, 13, 14, 15] }
```

Set the key of the motor

This method is used to set the security key of the motor. Here is the API for the security set method:

```
{
  "description" : "Set the current key of the motor",
  "path" : "security.set",
  "params" : [
    {
      "name" : "key", "type" : "array of 16 bytes"
    }
  ],
  "returns" : { "type" : "boolean", "description" : "true if the key
is successfully changed, false otherwise" }
  }
}
```

Example:

REQUEST

```
{ "method" : "security.set", "params" : {"key" : [0, 1, 2, 3, 4, 5, 6, 7,
8, 9, 10, 11, 12, 13, 14, 15] } }
```

RESPONSE

```
{ "result" : true, "id" : 1, "key" : }
```

The key is used to encrypt using AES CBC 128, data has to be padded with zero to have a size multiple of 16 bytes. The initialization vector with 16 random bytes shall be generated at each new UDP packet to send, and are localized at the beginning of the data.

Structure of a UDP packet: Input Vector (16 bytes) - Encrypted Data (Multiple of 16 bytes, zero padded)

All the Synergy commands sent on the UDP port shall be cyphered with the key.

Ping commands do not require encryption.

Between two successive UDP commands from one client the “id” shall evolve.

Error Code

When the reply contains the field result with the false value ({ "result" : false) an error code will be added with an associated code.

Example:

```
{ "result" : false, "error" : {"id": 32, "title":"Device is locked at
current position" } }
```

Table of errors

Error code (id)	Associated message (title)
01	Data out of range
16	Unknown message
17	Message Length Error
37	Feature not supported
32	Device is locked at current position
34	End limits are not SET
48	Device is in thermal protection
49	Device is in obstacle detection
36	Target position is out of range
38	In motion
33	Wrong position
35	No Preset Position SET
47	End limits are SET. Reset End Limits
40	Last Preset Position is reached
42	Low priority
43	Wink is in progress
45	Diagnostic is not available at this time
50	Incorrect or missing parameters
254	Unknown error %04x
255	Busy – Cannot process message